All models
Gemini 3
Nano Banana
Veo
Lyria 3
Lyria RealTime
Imagen
Text-to-speech
Embeddings
Robotics

Core capabilities

Text
Image
Video
Documents
Speech and audio
Thinking
Structured outputs

Try the new Gemini 3.1 Flash TTS Preview model for expressive, multilingual speech generation.

Text generation

The Gemini API can generate text output from text, images, video, and audio inputs.

Here's a basic example:

Python JavaScript Go Java REST Apps Script

```python
from google import genai

client = genai.Client()

response = client.models.generate_content(
    model="gemini-3-flash-preview",
    contents="How does AI work?"
)
print(response.text)
```

Thinking with Gemini

Gemini models often have "thinking" enabled by default which allows the model to reason before responding to a request.

Each model supports different thinking configurations which gives you control over cost, latency, and intelligence. For more details, see the thinking guide.

Python JavaScript Go Java REST Apps Script

```python
from google import genai
from google.genai import types

client = genai.Client()

response = client.models.generate_content(
    model="gemini-3-flash-preview",
    contents="How does AI work?",
    config=types.GenerateContentConfig(
        thinking_config=types.ThinkingConfig(thinking_level="low")
    ),
)
print(response.text)
```

System instructions and other configurations

You can guide the behavior of Gemini models with system instructions. To do so, pass a `GenerateContentConfig` object.

Python JavaScript Go Java REST Apps Script

```python
from google import genai
from google.genai import types

client = genai.Client()

response = client.models.generate_content(
    model="gemini-3-flash-preview",
    config=types.GenerateContentConfig(
        system_instruction="You are a cat. Your name is Neko."),
    contents="Hello there"
)

print(response.text)
```

The `GenerateContentConfig` object also lets you override default generation parameters, such as temperature.

> When using Gemini 3 models, we strongly recommend keeping the `temperature` at its default value of 1.0. Changing the temperature (setting it below 1.0) may lead to unexpected behavior, such as looping or degraded performance, particularly in complex mathematical or reasoning tasks.

Python JavaScript Go Java REST Apps Script

```python
from google import genai
from google.genai import types

client = genai.Client()

response = client.models.generate_content(
    model="gemini-3-flash-preview",
    contents=["Explain how AI works"],
    config=types.GenerateContentConfig(
        temperature=0.1
    )
)
print(response.text)
```

Refer to the `GenerateContentConfig` in our API reference for a complete list of configurable parameters and their descriptions.

Multimodal inputs

The Gemini API supports multimodal inputs, allowing you to combine text with media files. The following example demonstrates providing an image:

Python JavaScript Go Java REST Apps Script

```python
from PIL import Image
from google import genai

client = genai.Client()

image = Image.open("/path/to/organ.png")
response = client.models.generate_content(
    model="gemini-3-flash-preview",
    contents=[image, "Tell me about this instrument"]
)
print(response.text)
```

For alternative methods of providing images and more advanced image processing, see our image understanding guide. The API also supports document, video, and audio inputs and understanding.

Streaming responses

By default, the model returns a response only after the entire generation process is complete.

For more fluid interactions, use streaming to receive `GenerateContentResponse` instances incrementally as they're generated.

Python JavaScript Go Java REST Apps Script

```python
from google import genai

client = genai.Client()

response = client.models.generate_content_stream(
    model="gemini-3-flash-preview",
    contents=["Explain how AI works"]
)
for chunk in response:
    print(chunk.text, end="")
```

Multi-turn conversations (chat)

Our SDKs provide functionality to collect multiple rounds of prompts and responses into a chat, giving you an easy way to keep track of the conversation history.

> **Note:** Chat functionality is only implemented as part of the SDKs. Behind the scenes, it still uses the `generateContent` API. For multi-turn conversations, the full conversation history is sent to the model with each follow-up turn.

Python JavaScript Go Java REST Apps Script

```python
from google import genai

client = genai.Client()
chat = client.chats.create(model="gemini-3-flash-preview")

response = chat.send_message("I have 2 dogs in my house.")
print(response.text)

response = chat.send_message("How many paws are in my house?")
print(response.text)

for message in chat.get_history():
    print(f'role - {message.role}',end=": ")
    print(message.parts[0].text)
```

Streaming can also be used for multi-turn conversations.

Python JavaScript Go Java REST Apps Script

```python
from google import genai

client = genai.Client()
chat = client.chats.create(model="gemini-3-flash-preview")

response = chat.send_message_stream("I have 2 dogs in my house.")
for chunk in response:
    print(chunk.text, end="")

response = chat.send_message_stream("How many paws are in my house?")
for chunk in response:
    print(chunk.text, end="")

for message in chat.get_history():
    print(f'role - {message.role}', end=": ")
    print(message.parts[0].text)
```

Prompting tips

Consult our prompt engineering guide for suggestions on getting the most out of Gemini.

What's next

- Try Gemini in Google AI Studio.
- Experiment with structured outputs for JSON-like responses.
- Explore Gemini's image, video, audio and document understanding capabilities.
- Learn about multimodal file prompting strategies.

On this page
Thinking with Gemini
System instructions and other configurations
Multimodal inputs
Streaming responses
Multi-turn conversations (chat)
Prompting tips
What's next

Was this helpful?
Send feedback

Except as otherwise noted, the content of this page is licensed under the Creative Commons Attribution 4.0 License, and code samples are licensed under the Apache 2.0 License. For details, see the Google Developers Site Policies. Java is a registered trademark of Oracle and/or its affiliates.

Last updated 2026-01-16 UTC.